

## 六、机械臂取餐实验

### 1. 实验描述

在智能餐饮场景中，商服机器人按照上次实训所讲解的逻辑与方法完成了自主点餐并生成了相应的订单，接下来就需要自主导航至取餐台，根据订单完成智能取餐的任务。

本次实验会对智能取餐这一单独的功能进行测试，首先要基于上位机软件——调试助手完成机械臂参数设置与动作示教；然后操作商服机器人至取餐台前，启用深度摄像头识别取餐台上订单所需的食物，自主调整位姿正对食物，再使用机械臂夹取对应的食物；夹取过程中会结合机械臂中的 RGB 摄像头调整抓取的角度，直到成功抓取食物。本次实验我们会模拟“智能取餐”这一步骤环节的核心功能。

### 2. 实验目标

- （1）掌握智能取餐的代码编程逻辑，会对其中的代码和参数进行调整或优化，并实操验证；
- （2）会操作上位机软件，完成对机械臂相关参数的设置，会对机械臂进行示教；
- （3）会发布指令，操作商服机器人控制机械臂进行物品抓取等智能取餐动作。

### 3. 实验步骤

#### 步骤 1：认识智能取餐的工程文件

通过连接网线、以太网设置等方式将个人 PC 远程连接商服机器人，在个人 PC 中使用 VSCode 通过远程连接的方式打开商服机器人中的工程文件 `sebot-t710-competition`，在路径“`/root/workspace/sebot-t710-competition/sebot_catering/src/sebot_catering/src/picking.cpp`”与“`/root/workspace/sebot-t710-competition/sebot_catering/src/sebot_catering/unit/pick.cpp`”找到本次智能取餐实验所需的工程代码文件 `picking.cpp` 与 `pick.cpp`。

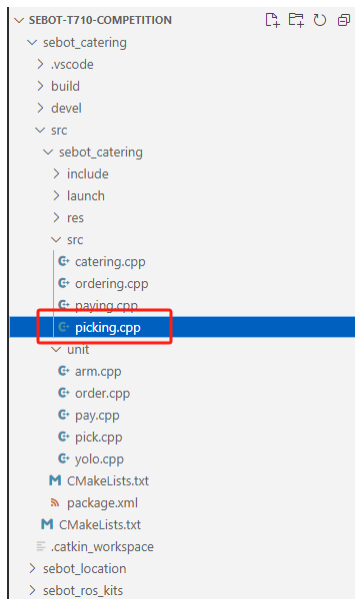


图 6-1 智能取餐脚本代码 01 所在路径

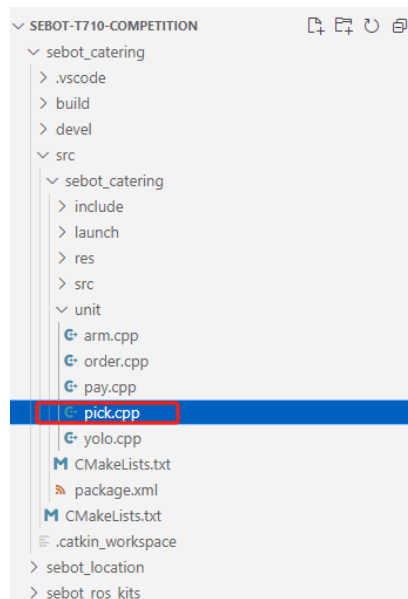


图 6-2 智能取餐脚本代码 02 所在路径

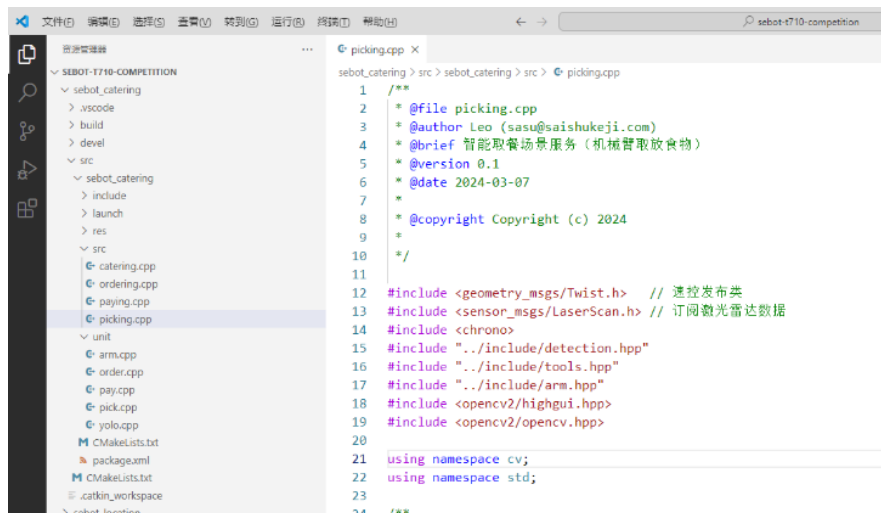


图 6-3 智能取餐功能代码文件 picking.cpp 界面

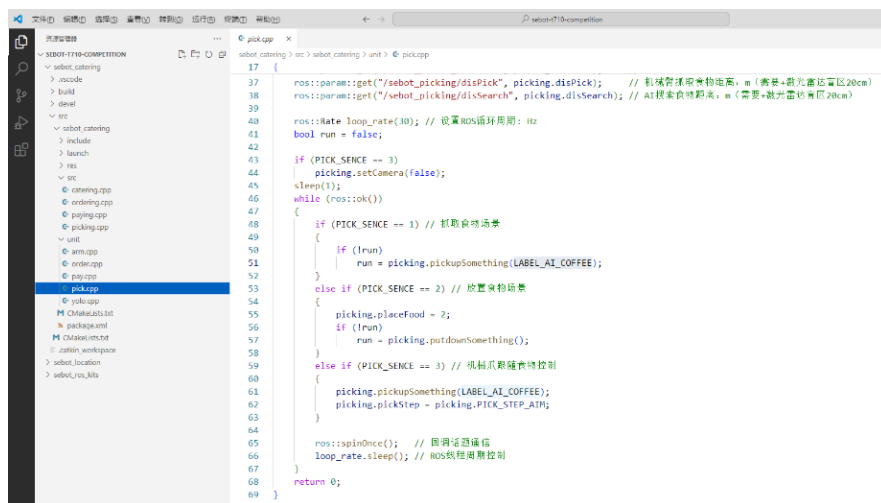


图 6-4 启动智能取餐功能代码 pick.cpp

接下来我们基于上述两个代码文件，对代码的详细逻辑进行讲解。

## 步骤 2：了解智能取餐核心功能

智能取餐功能大致可以归类为如下几个模块，如下图所示。

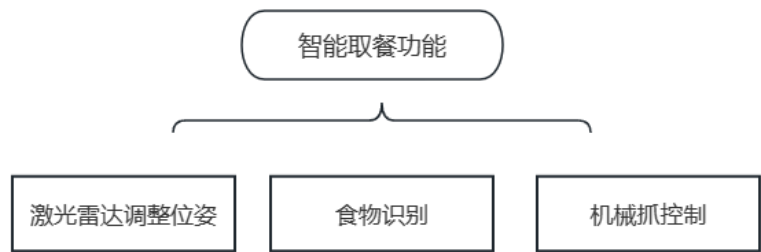


图 6-5 功能结构

在智能餐饮场景中，商服机器人已经完成了场地建图、自主点餐的任务，此时商服机器人需要根据订单中所体现的订单信息去取餐台将食物取回来并递放至相应的餐桌上。

假设商服机器人已经移动到对应的取餐台面前，并且启动了本次的单元测试程序——智能取餐功能，其逻辑如下：

第一步，机械臂示教。需要对机械臂进行动作示教，模拟抬起、抓取、放下等姿势，让机械臂记住该操作流程，后续商服机器人在进行食物抓取或者食物放置时均会按照该动作完成。

第二步，商服机器人调整位姿，识别食物。商服机器人会在桌子前根据激光雷达的数据调整位姿，主要是调整与桌子之间的距离。商服机器人需要正对桌子，与此同时，开启商服机器人上的深度摄像头去识别取餐台中的食物。在目标识别过程中，要保证食物在摄像头获取的图像的正中间。

第三步，抓取食物。成功识别食物后，商服机器人会继续调整位姿，保证与食物居中对齐，启动机械臂抓取动作，开启机械臂上的 RGB 摄像头再次寻找食物并使其图像位于正中央，保证机械臂可以准确无误的抓取到食物。

因为是单元测试阶段，只单纯测试智能取餐功能，所以此环节将在一张桌子上执行整个取餐过程中食物抓取涉及到的机械臂关键功能。抓取到食物后的商服机器人会后退并收缩机械臂，执行收缩动作，机械爪会夹取着食物随机械臂收缩动作回到商服机器人的内部货架上方。

本实验中仅包含智能取餐上述的几个步骤中所提到的核心功能，最终会通过一个单元测

试的代码去单次启动上述智能取餐环节的功能，让我们直观的体验智能取餐是怎样的行为与过程。

### 步骤 3：理解智能取餐代码逻辑

代码环节一共包含三个主要部分：① 函数库的引入；② 私有类功能；③ 共有类功能；其中私有类功能是面向底层的内部逻辑；共有类功能是面向对象，属于调用部分的上层逻辑，具体内容我们在下文详细讲解。代码讲解过程中，只介绍主要代码功能，因为是单元测试的功能代码，有些函数功能可能会在之前的实训中有所提及，限于篇幅，重复的功能代码在本次实训中不重复讲解，一些偏辅助类的功能代码，在本次实训中同样不重复讲解，详细代码需要自己翻看之前的实训手册或者查看源代码。

#### 1、导入库函数（src/picking.cpp）

- （1）geometry\_msgs/Twist.h 头文件，用于发布速度控制指令。
- （2）sensor\_msgs/LaserScan.h 头文件，用于订阅激光雷达数据。
- （3）chrono 头文件，用于处理时间相关的操作。
- （4）detection.hpp 头文件，文件路径为../include/detection.hpp，用于完成目标检测与识别相关功能；
- （5）tools.hpp 头文件，文件路径为../include/tools.hpp，用于一些工具的使用；
- （6）arm.hpp 头文件，文件路径为../include/arm.hpp，用于机械臂功能的使用；
- （7）opencv2/highgui.hpp 头文件，用于实现图形用户界面相关的操作；
- （8）opencv2/opencv.hpp 头文件，用于实现计算机视觉相关的操作。

#### 2、智能取餐场景类——私有部分（class Picking）

##### （1）私有部分变量定义

首先定义了一个私有部分，包括了以下变量的定义：

- ① rosNode：ROS 节点类的实例，用于与 ROS 系统进行通信。
- ② detection：指向 Detection 类实例的智能指针，用于进行 AI 目标检测。
- ③ captureDeep：摄像头深度图像采集类的实例，用于获取深度图像。
- ④ captureRgb：摄像头 RGB 图像采集类的实例，用于获取 RGB 图像。
- ⑤ subLaser：激光雷达话题的订阅者，用于接收激光雷达数据。
- ⑥ pubCmd：速控话题的发布者，用于发布速度控制命令。

- ⑦ counter: 计数器变量, 用于计算程序运行的次数。
- ⑧ orienOffset: 姿态方向校正误差变量, 用于调整目标姿态的方向。
- ⑨ talon: 机械臂控制类的实例, 用于控制机械臂的运动。

## (2) 定义激光雷达类

定义该类的目的是为了进行激光雷达相关功能操作, 该类的定义如下:

```
class Rplidar
```

定义了一个名为 **Rplidar** 的类, 并实例化了一个名为 **rplidar** 的对象。类 **Rplidar** 有以下成员变量:

- ① angle: 一个 float 类型的变量, 表示激光雷达的探测角度, 单位为弧度。该值设定为 1.57, 表示激光雷达在机器人前方的探测角度为 1.57 弧度。
- ② angleIncrement: 一个 float 类型的变量, 表示间隔取点的度数, 单位为弧度。该值设定为 0.087, 表示每隔大约 5° 进行一次数据采集。
- ③ rangesLeft: 一个 vector<float>类型的变量, 用于存储左侧距离数据。
- ④ rangesRight: 一个 vector<float>类型的变量, 用于存储右侧距离数据。

接下来, 通过 **Rplidar** 类实例化了一个对象 **rplidar**, 表示一个激光雷达。通过这个对象, 可以访问并使用 **Rplidar** 类中定义的成员变量和方法。

## (3) 接收激光雷达数据 (void getLaser())

这是一个用于接收激光雷达数据的函数。代码的作用是接收激光雷达的数据, 并根据数据的参数和设定的步长筛选出感兴趣的数据点, 并将其存储到对象的成员变量中, 以便后续的处理和使用。

## (4) 获取机械爪夹取食物的间距 (单位: cm): (void getClawPosition())

**getClawPosition** 的函数接受一个 **food** 参数 (字符串类型) 作为输入, 并返回一个浮点数类型的值。函数通过对输入的 **food** 参数进行比较, 判断食物的类型, 并根据不同的食物类型, 返回对应的机械爪位置。

具体的判断逻辑如下:

如果 **food** 等于 **LABEL\_AI\_COLA** (可乐), 函数返回浮点数 4.3。

如果 **food** 等于 **LABEL\_AI\_COFFEE** (咖啡), 函数返回浮点数 6.0。

如果 **food** 等于 **LABEL\_AI\_BURGER** (汉堡), 函数返回浮点数 6.0。

如果 **food** 等于 **LABEL\_AI\_CAKE** (瑞士卷), 函数返回浮点数 3.0。

如果 **food** 等于 **LABEL\_AI\_CHEESE** (奶酪), 函数返回浮点数 2.0。

如果 food 等于 LABEL\_AI\_SUNDAE (圣代), 函数返回浮点数 4.4。

如果以上条件均不满足, 函数返回浮点数 5.0。

这个函数的作用是根据不同的食物类型, 返回机械爪收缩后的距离, 以便正确抓取食物。

(代码较短已省略, 详细内容请查看源代码)

#### (5) 控制机械爪搜索食物位置 (void clawFindFoods())

clawFindFoods 的函数接受一个 food 参数 (字符串类型) 作为输入, 用于控制机械爪搜索食物的位置。函数的目的是根据传入的食物类型, 在 RGB 图像中搜索食物的位置, 并根据搜索结果控制机械爪的运动。如果成功检测到目标食物, 则启用机械爪的位置控制, 否则禁用位置控制。

该函数内部的逻辑如下:

① 调用 searchFoodRgb 函数, 并传入 food 参数, 以获取食物的中心点位置。返回的中心点位置保存在 Center 类型的变量 center 中。

② 如果检测到有效的目标食物 (中心点的 x 坐标小于 COLSIMAGE 且 y 坐标小于 ROWSIMAGE), 则将 pidClawX.enable 和 pidClawY.enable 设置为 true, 表示启用机械爪的 x 和 y 方向的位置控制。

③ 否则, 将 pidClawX.enable 和 pidClawY.enable 设置为 false, 表示禁用机械爪的位置控制。

④ 调用 clawControl 函数, 并传入 center 参数, 用于控制机械爪的运动。

```
/**
 * @brief 控制机械爪搜索食物位置
 *
 * @param food
 */
void clawFindFoods(string food)
{
    Center center = searchFoodRgb(food);

    if (center.x < COLSIMAGE && center.y < ROWSIMAGE) // 检测到有效的
目标食物
    {
        pidClawX.enable = true;
```

```

        pidClawY.enable = true;
    }
    else
    {
        pidClawX.enable = false;
        pidClawY.enable = false;
    }

    clawControl(center); // 机械爪运动控制
}

```

### 3、智能取餐场景类——公有部分（class Picking）

#### （1）公有部分变量定义

- ① pidPose: PID 姿态控制器，用于控制机器人的方向（角度）。
- ② pidDis: PID 姿态控制器，用于控制机器人与目标的距离（X 轴）。
- ③ pidLocal: PID 姿态控制器，用于控制机器人在 Y 轴上的定位。
- ④ pidClawX: PID 姿态控制器，用于控制机械爪在 X 轴上的方向。
- ⑤ pidClawY: PID 姿态控制器，用于控制机械爪在 Y 轴上的方向。
- ⑥ debug: 调试使能标志，用于启用或禁用 AI 绘制和显示功能。
- ⑦ findFood: AI 搜索到目标食物的标志，用于指示是否已经找到目标食物。
- ⑧ disSearch: AI 搜索食物的距离，单位为米。
- ⑨ disPick: 机械臂抓取食物的距离，单位为米。
- ⑩ disClaw: 机械爪夹取食物的距离，单位为米。
- ⑪ placeFood: 食物放置位置（餐桌）的选项，可以为 1（中间）、2（左边）或 3（右边）。

这些变量用于控制机器人和机械爪的运动参数、调试功能以及食物的搜索和放置位置等相关设置。

#### （2）智能取餐任务流程

如下内容包含了智能取餐任务的流程和相关的初始化函数和析构函数。

- ① enum PickStep: 定义了一个枚举类型 PickStep，表示智能取餐任务的不同步骤。包括任务开始、机器人姿态校正、食物搜索、机器人向前移动、机械爪瞄准食物、食物抓取、

机器人重定位和智能取餐结束这几个步骤。

② `PickStep pickStep`: 智能取餐任务流程的变量，初始值为 `PICK_STEP_START`，即任务开始。

③ `Picking()`: 构造函数，进行一些初始化操作，包括获取功能包文件的根路径、初始化 NPU 和 AI 模型、设置摄像头、订阅激光雷达话题和发布速控话题。还初始化了一些 PID 控制器的参数，并在构造函数最后进行了 1 秒的延时。

④ `Picking()`: 析构函数，用于释放资源，包括停止机器人运动、释放深度图像和 RGB 图像采集类的资源。（如下部分代码已省略，详细内容请查看源代码）

```
/**
 * @brief 智能取餐任务流程
 *
 */
enum PickStep
{
    PICK_STEP_START = 0, // 任务开始
    .....
    PICK_STEP_END,      // 智能取餐结束
};

PickStep pickStep = PickStep::PICK_STEP_START; // 智能取餐任务流程

Picking()
{
    string pathPkg = ros::package::getPath("sebot_catering"); //
    功能包文件根路径

    detection = make_shared<Detection>(pathPkg + "/res/model"); //
    初始化 NPU 及 AI 模型

    detection->score = 0.4; // AI 检测置信度

    // 摄像头初始化
    setCamera(true);
```



```

.....

};

~Picking()

{

    robotCtrl(0, 0, 0); // 机器人运动控制

    captureDeep.release();

    captureRgb.release();

};

```

这个公用类部分代码用于控制智能取餐任务的流程，包括姿态校正、食物搜索、机器人运动、机械爪控制等，并具有一些初始化和资源释放的功能。

### (3) 选择相机通道 (void setCamera())

此函数用于选择相机通道。函数接受一个 depth 参数（布尔类型），用于判断是否选择深度相机。如果 depth 为 true，则选择深度相机；如果 depth 为 false，则选择 RGB 相机。

如果选择深度相机，函数会打开相应的摄像头设备文件/dev/deepCamera，并设置图像的宽度与高度为特定数值。

如果选择 RGB 相机，函数会释放深度图像的采集类资源，并进行 1 秒的延时，然后初始化和打开 RGB 摄像头设备文件/dev/rgbCamera，同时也设置图像的宽度和高度。（如下部分代码已省略，详细内容请查看源代码）

```

/**

 * @brief 选择相机通道

 *

 * @param depth 是否选择深度相机

 */

void setCamera(bool depth)

{

    if (depth) // 深度相机

    {

        .....

    }

    else // RGB 相机

```

```

{
    .....
}

}

```

该函数可以根据需要选择使用深度相机或 RGB 相机，并进行相应的初始化和配置。

#### （4）机器人运动控制（void robotCtrl ()）

此函数用于控制机器人的运动。函数接受三个参数：linearX 表示 X 轴线速度，linearY 表示 Y 轴线速度，angular 表示 Z 轴角速度。函数通过创建一个 geometry\_msgs::Twist 类型的对象 msgCmd 来保存速度控制数据，然后将传入的线速度和角速度赋值给 msgCmd 的对应成员变量。最后，函数通过发布 msgCmd 对象到速度控制话题来实现机器人的运动控制。

（此部分代码部分与实训 5 所讲述代码相同，详细内容请查看源代码）

#### （5）机器人姿态控制（距离+方向+位置，void poseControl ()）

poseControl 函数是一个用于控制机器人姿态的函数。综合使用方向、距离和位置控制功能，通过 PID 控制器对机器人姿态进行调整，以达到期望的运动姿态。

如下是函数的大致代码思路：

第一步，判断激光雷达数据的左右激光点集的大小是否满足条件。如果满足条件，通过计算左右激光点集的中位数之差来进行方向控制，并根据 PID 控制器的输出得到角速度。

第二步，根据左右激光点集的中位数之和除以 2 来进行距离控制，通过 PID 控制器得到线速度。

第三步，通过 PID 控制器对位置进行控制，得到侧向速度。

第四步，调用 robotCtrl 函数将计算得到的线速度、侧向速度和角速度作为参数，实现机器人的运动控制。（代码部分与实训 5 所讲述代码相同，详细请查看源代码）

#### （6）机械抓运动控制（void clawControl()）

clawControl 函数主要用于控制机械爪的运动。函数接受一个 center 参数，表示食物的中心点位置，类型为 Center 结构体。

首先，如果 pidClawX.enable 为 true，表示机械爪在 X 轴上的位置控制被启用。函数会设置 pidClawX 的参考值(ref)为 0，将食物中心点的 X 坐标作为反馈值(feedBack)，并调用 pdController 函数对 pidClawX 进行 PD 控制。最后，函数会输出调试信息，包括机械爪 X 轴的误差(pidClawX.feedBack)和输出值(pidClawX.out)。

接着，如果 pidClawY.enable 为 true，表示机械爪在 Y 轴上的位置控制被启用。函数会设

置 `pidClawY` 的参考值(ref)为 0，将食物中心点的 Y 坐标作为反馈值(feedBack)，并调用 `pdController` 函数对 `pidClawY` 进行 PD 控制。最后，函数会输出调试信息，包括机械爪 Y 轴的误差(`pidClawY.feedBack`)和输出值(`pidClawY.out`)。

最后，函数调用 `talon` 对象的 `setClawMotion` 函数，将经过控制的机械爪 X 轴和 Y 轴的输出值(`-pidClawX.out` 和 `pidClawY.out`)传递给机械爪，实现机械爪的运动控制。

```
/**
 * @brief 机械爪运动控制
 *
 */
void clawControl(Center center)
{
    if (pidClawX.enable)
    {
        pidClawX.ref = 0;
        pidClawX.feedBack = center.x;
        pdController(pidClawX);
        ROS_ERROR_STREAM("error-claw-x:" +
to_string(pidClawX.feedBack) + " out:" + to_string(pidClawX.out));
    }

    if (pidClawY.enable)
    {
        pidClawY.ref = 0;
        pidClawY.feedBack = center.y;
        pdController(pidClawY);
        ROS_ERROR_STREAM("error-claw-y:" +
to_string(pidClawY.feedBack) + " out:" + to_string(pidClawY.out));
    }

    talon.setClawMotion(-pidClawX.out, pidClawY.out); // 机械爪运动控
```

```
制
```

```
}
```

该函数的功能是根据食物的中心点位置，控制机械爪在 X 轴和 Y 轴上的运动，达到对食物进行抓取或放置的目的。同时，函数会输出相关的调试信息。

#### (7) 搜索食物——深度相机 (Center searchFoodDepth())

`searchFoodDepth` 函数用于在深度图像中搜索指定的食物。函数接受一个 `food` 参数，表示食物的 AI 标签，类型为字符串。函数返回一个 `Center` 类型的对象，表示搜索到的食物的控制中心。

函数内部的大致代码逻辑如下：

第一步，函数通过深度相机采集图像数据，如果无法读取到图像数据，则返回空的 `Center` 类型的对象。

第二步，函数调用 AI 模型进行推理，获取检测结果。遍历检测结果，筛选出标签与指定食物相同、宽度小于 80、高度小于 120 的物体，并将筛选结果保存到 `results` 向量中。

第三步，定义一个 `result` 对象，用于保存具有最高评分的目标物体。遍历 `results` 向量，根据条件选择最底部的目标物体作为 `result` 对象。

第四步，根据 `result` 对象的标签与指定食物相同，计算控制中心的坐标。`center.x` 的值为 `result.x + result.width / 2 - COLSIMAGE / 2`，表示相对于图像中心的 X 轴偏移量；`center.y` 的值为 `result.y + result.height / 2 - ROWSIMAGE / 2`，表示相对于图像中心的 Y 轴偏移量。

第五步，如果处于调试模式下 (`debug` 为 `true`)，则绘制目标框和标签，并显示图像窗口。

第六步，返回 `Center` 类型的对象。(部分代码已省略，详细内容请查看源代码)

```
/**
 * @brief 搜索食物
 *
 * @param food 食物 AI 标签
 * @return Center 返回控制中心: center
 */
Center searchFoodDepth(string food)
{
    Center center;
```

```

    Mat img;

    if (!captureDeep.read(img)) // 相机采集数据

        return center;

    // 启动 AI 推理

    detection->inference(img); // NPU 推理

    .....

    return center;

}

```

该函数的功能是在深度图像中搜索指定的食物，并返回食物的 **Center** 类型的对象。函数通过调用 **AI** 模型进行目标检测，根据检测结果筛选出符合条件的目标物体，并计算出 **Center** 类型的对象的坐标。函数还具有调试模式，可以绘制目标框和标签，并显示图像。

#### （8）搜索食物——RGB 相机（**Center searchFoodRgb()**）

**searchFoodRgb** 函数主要用于在机械臂配置的 **RGB** 摄像头图像中搜索指定的食物，与深度相机搜索食物的方式较为相似，这两个函数的主要作用都是为了准确的搜索食物所在的位置，深度摄像头是放置于商服机器人中，**RGB** 摄像头是放置于机械臂上。函数接受一个 **food** 参数，表示食物的 **AI** 标签，类型为字符串。函数返回一个 **Center** 类型的对象，表示搜索到的食物。

函数代码逻辑如下：

第一步，函数通过 **RGB** 相机采集图像数据，如果无法读取到图像数据，则返回空的 **Center** 类型的对象。

第二步，函数调用 **AI** 模型进行推理，获取检测结果。遍历检测结果，筛选出标签与指定食物相同的物体，并将筛选结果保存到 **results** 向量中。

第三步，定义一个 **result** 对象，用于保存评分最高的目标物体。遍历 **results** 向量，选取评分最高的目标物体作为 **result** 对象。

第四步，根据 **result** 对象的标签与指定食物相同，计算控制中心的坐标。**center.x** 的值为 **result.x + result.width / 2 - COLSIMAGE / 2**，表示相对于图像中心的 **X** 轴偏移量；**center.y** 的值为 **result.y + result.height / 2 - ROWSIMAGE / 2**，表示相对于图像中心的 **Y** 轴偏移量。

第五步，如果处于调试模式下（**debug** 为 **true**），则绘制目标框和标签，并显示图像窗口。

第六步，返回控制中心。（部分代码已省略，详细内容请查看源代码）

```
/**
 * @brief 搜索订单内容（食物）
 *
 * @return vector<PredictResult>
 */
vector<PredictResult> searchFoods()
{
/**
 * @brief 搜索食物(RGB 相机)
 *
 * @param food 食物 AI 标签
 * @return Center 返回控制中心: center
 */
Center searchFoodRgb(string food)
{
    Center center;

    Mat img;

    if (!captureRgb.read(img)) // 相机采集数据 RGB
        return center;

    // 启动 AI 推理
    detection->inference(img); // NPU 推理
    .....
    return center;
}
```

该函数的功能是在 RGB 图像中搜索指定的食物，并返回食物的 Center 类型的对象。函数通过调用 AI 模型进行目标检测，根据检测结果筛选出符合条件的目标物体，并计算出 Center 类型的对象的坐标。函数还具有调试模式，可以绘制目标框和标签，并显示图像。

（9）智能取餐(食物抓取场景 bool pickupSomething(string food))

此部分代码较长，我们对此部分的主要代码思路和模块功能进行概要介绍。这段代码实现了一个智能的食物抓取场景。它包含以下几个主要步骤：

首先，变量的声明，一共声明了 6 个静态变量：

- ① count：用于 AI 检测计数的变量，初始值为 0。
- ② scene：用于图像场计数的变量，初始值为 0。
- ③ thisTime：用于存储当前系统时间的变量，初始值为获取的系统时间。
- ④ countLeft：用于左移动计数的变量，初始值为 0。
- ⑤ countRight：用于右移动计数的变量，初始值为 0。
- ⑥ searchLeft：用于指示是否向左移动搜索目标的布尔变量，初始值为 true。

在函数内部，使用了一个 switch 语句来根据变量 pickStep 的不同值执行不同的操作。在每个 case 中，针对不同的 pickStep 值，进行了不同的操作，流程图如下所示：

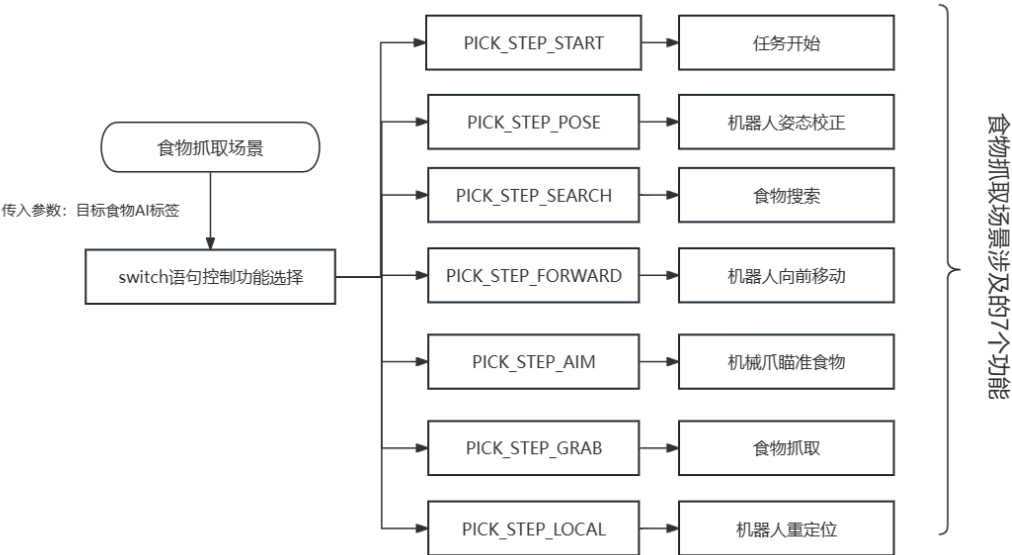


图 6-6 食物抓取场景功能框架图

一共包含七个部分的功能：

- ① PICK\_STEP\_START（任务开始）：机器人会伸展机械臂，并将机械爪张开到 9cm 的固定位置。这为后续的抓取动作做好准备。
- ② PICK\_STEP\_POSE（机器人姿态校正）：机器人使用深度相机搜索目标食物，并通过 PID 控制调整自身姿态，使机械臂能够精准对准食物。如果找到目标食物并调整姿势成功，则切换到下一步，否则切换到搜索步骤。
- ③ PICK\_STEP\_SEARCH（食物搜索）：如果之前未能找到目标食物，机器人会通过左右移动的方式进行搜索。找到目标食物后，继续调整姿势直至能够精准对准。

④PICK\_STEP\_FORWARD（机器人向前移动）:机器人缓慢向前移动，同时使用 RGB 相机跟踪食物位置，控制机械爪保持对准。直到达到预设的抓取距离，或者超时退出。

⑤ PICK\_STEP\_AIM（机械爪瞄准食物）:机械爪会精确瞄准目标食物，通过 PID 控制调整自身位置和角度。直到定位准确或超时，即可进入最终抓取步骤。

⑥PICK\_STEP\_GRAB（食物抓取）:机器人移动到预设的抓取距离，然后根据食物类型调整机械爪的夹取位置，执行抓取动作。抓取完成后，切换到机器人重新定位的步骤。

⑦PICK\_STEP\_LOCAL（机器人重定位）:机器人缓慢后退到合适的导航距离，为下一次抓取任务做好准备。完成整个食物抓取的流程。

最后，如果 pickStep 的变量标志位==PICK\_STEP\_END（任务结束），则返回 True 的标识，否则返回 False 的标识，返回类型为布尔类型。（代码过长，不放置于文中）

#### （10）放置食物（bool putdownSomething()）

此部分代码量也较大，描述其主要代码逻辑，详细代码请查看源文件，如下是该函数的大致逻辑。这段代码是一个名为 putdownSomething() 的函数，它的作用是完成放置食物的任务流程。在函数内部，使用了一个 switch 语句来根据变量 pickStep 的不同值执行不同的操作。在每个 case 中，针对不同的 pickStep 值，进行了不同的操作：

① PICK\_STEP\_START：任务开始阶段，记录当前系统时间并将 pickStep 切换到 PICK\_STEP\_POSE。

② PICK\_STEP\_POSE：机器人姿态校正阶段。首先关闭位置 PID 控制，设置机械臂抓取食物的目标距离。然后检测当前姿态校正的状态，如果满足条件则切换到 PICK\_STEP\_GRAB 并执行一系列动作，包括:设置机械臂动作为放置食物、控制机器人停止运动等待机械臂伸展完成、初始化机械爪、记录当前系统时间；如果不满足条件，则调用姿态控制函数 poseControl()。

③PICK\_STEP\_GRAB: 食物放置阶段。设置机械臂放置食物的目标距离，调用 poseControl() 进行姿态控制。然后检测当前姿态校正的状态，如果满足条件则切换到 PICK\_STEP\_LOCAL 并执行一些动作，包括:、控制机器人停止运动、设置机械爪的关节位置以放置食物、等待食物放置完成；最后设置机器人后退的目标距离，并记录当前系统时间。

④PICK\_STEP\_LOCAL：机器人重定位阶段。检测当前的姿态校正状态，如果满足条件则将 pickStep 切换到 PICK\_STEP\_END，并设置机械臂执行收缩动作；如果不满足条件，则继续调用 poseControl() 进行姿态控制。



最后，该函数会根据 `pickStep` 的值返回 `true` 或 `false`。`putdownSomething()`函数实现了一个食物放置的任务流程，包括机器人姿态校正、食物放置以及机器人重定位等步骤。

#### 4、单元测试代码（`unit/pick.cpp`）（此部分需要填充代码）

此部分的代码为了单独测试智能取餐功能而设置，用于检测商服机器人在此环节中的代码功能实现效果。

##### （1）函数库的引入

`#include "../src/picking.cpp"`为引入上一个环的 `picking.cpp` 文件，因为主要的核心代码功能都在此功能类中。

##### （2）主程序部分（注意：此部分需要代码填空）

此部分为调用智能配送进行单元测试的主程序部分，主程序部分大致逻辑如下：

①通过 `ros::init` 函数初始化 ROS 节点，并命名为“`sebot_picking`”。然后实例化了一个 `Picking` 对象 `picking`，即智能取餐类。（此环节需要代码填充）

```
ros::init(argc, argv, "sebot_picking");  
  
Picking picking; // 实例化智能取餐
```

② 通过 `ros::param::get` 函数获取配置参数值。这些参数包括方向控制 `PID` 的参数、距离控制 `PID` 的参数、位置控制 `PID` 的参数、机械爪 `PID` 的参数、调试标志位、机械爪夹取食物距离、机械臂抓取食物距离和 `AI` 搜索食物距离等。这些参数的值是在程序运行之前预先设置好的，通过设置不同的参数值可以调整机器人的控制参数和行为。

```
ros::param::get("/sebot_picking/pidPoseKp", picking.pidPose.kP);  
// 方向控制 PID  
  
ros::param::get("/sebot_picking/pidPoseKi", picking.pidPose.kI);  
ros::param::get("/sebot_picking/pidPoseKd", picking.pidPose.kD);  
ros::param::get("/sebot_picking/pidDisKp", picking.pidDis.kP); //  
距离控制 PID  
  
ros::param::get("/sebot_picking/pidDisKi", picking.pidDis.kI);  
ros::param::get("/sebot_picking/pidDisKd", picking.pidDis.kD);  
ros::param::get("/sebot_picking/pidLocalKp", picking.pidLocal.kP);  
// 位置控制 PID  
  
ros::param::get("/sebot_picking/pidLocalKi", picking.pidLocal.kI);
```

```

    ros::param::get("/sebot_picking/pidLocalKd", picking.pidLocal.kD);
    ros::param::get("/sebot_picking/pidClawXKp", picking.pidClawX.kP);
// 机械爪 PID
    ros::param::get("/sebot_picking/pidClawXKi", picking.pidClawX.kI);
    ros::param::get("/sebot_picking/pidClawXKd", picking.pidClawX.kD);
    ros::param::get("/sebot_picking/pidClawYKp", picking.pidClawY.kP);
// 机械爪 PID
    ros::param::get("/sebot_picking/pidClawYKi", picking.pidClawY.kI);
    ros::param::get("/sebot_picking/pidClawYKd", picking.pidClawY.kD);
    ros::param::get("/sebot_picking/debug", picking.debug);
    ros::param::get("/sebot_picking/disClaw", picking.disClaw); //
机械爪夹取食物距离: m (需要+激光雷达盲区 20cm)
    ros::param::get("/sebot_picking/disPick", picking.disPick); //
机械臂抓取食物距离: m (需要+激光雷达盲区 20cm)
    ros::param::get("/sebot_picking/disSearch", picking.disSearch); //
AI 搜索食物距离: m (需要+激光雷达盲区 20cm)

```

③ 创建了一个循环频率为 30Hz 的 `ros::Rate` 对象 `loop_rate`。

```

ros::Rate loop_rate(30); // 设置 ROS 循环周期: Hz

bool run = false;

```

④代码检查变量 `PICK_SENCE` 的值是否为 3。如果是,会调用 `picking.setCamera(false)` 函数来关闭相机,之后等待 1 秒。

```

if (PICK_SENCE == 3)

    picking.setCamera(false);

sleep(1);

```

⑤进入程序的执行部分,用于完成智能取餐场景。

第一步,代码会进入一个无限循环,只有当 ROS 节点还在运行时才会继续循环。

第二步,如果 `PICK_SENCE` 的值为 1,则调用函数 `picking.pickupSomething(LABEL_AI_COFFEE)`;将食物抓起来。如果之前没有进行过抓取动作 (`run` 为 `false`),则将 `run` 的值设置为 `true`。

第三步,如果 `PICK_SENCE` 的值为 2,则将 `picking.placeFood` 设置为 2,并调用

`picking.putdownSomething()`;函数将食物放下。如果之前没有进行过放下动作（`run` 为 `false`），则将 `run` 的值设置为 `true`。

第四步，如果 `PICK_SENCE` 的值为 3，则调用 `picking.pickupSomething(LABEL_AI_COFFEE)`;函数将食物抓起来，并将 `picking.pickStep` 的值设置为 `picking.PICK_STEP_AIM`。主要作用是用于机械爪跟随食物控制

在每次循环的结尾，代码调用 `ros::spinOnce()`;来处理回调话题通信，然后调用 `loop_rate.sleep()`;来控制 ROS 线程的周期。

最后，代码会返回 0，表示正常结束。

```
while (ros::ok())
{
    if (PICK_SENCE == 1) // 抓取食物场景
    {
        if (!run)
            run = picking.pickupSomething(LABEL_AI_COFFEE);
    }

    else if (PICK_SENCE == 2) // 放置食物场景
    {
        picking.placeFood = 2;
        if (!run)
            run = picking.putdownSomething();
    }

    else if (PICK_SENCE == 3) // 机械爪跟随食物控制
    {
        picking.pickupSomething(LABEL_AI_COFFEE);
        picking.pickStep = picking.PICK_STEP_AIM;
    }

    ros::spinOnce(); // 回调话题通信
    loop_rate.sleep(); // ROS 线程周期控制
}
```

```
return 0;
```

### 步骤 4：机械臂示教动作执行

在进行实操之前，首先我们要了解机械臂的使用，并且对机械臂完成示教操作，这样后续商服机器人在执行取餐任务时才会按照我们示教的动作、角度去完成抓取任务。

第一步，安装“六轴协作机械臂调试助手.exe”。可参考机械臂说明书，基于提供的各项安装包，在自己的电脑上独立安装“ [Talon]六轴协作机械臂调试助手安装包”。安装完成后重启计算机。

| 名称                                                                                                         | 修改日期            |
|------------------------------------------------------------------------------------------------------------|-----------------|
|  [Talon]六轴协作机械臂_Setup.exe | 2024/3/25 16:14 |

图 6-7 [Talon]六轴协作机械臂调试助手安装包



图 6-8 [Talon]六轴协作机械臂调试助手

第二步，认识[Talon]六轴协作机械臂调试助手软件界面。界面一共分为 6 个部分：

- ①机械臂的示意图，图中对每个电机的位置进行标号并实时显示当前机械臂姿态每个电机的弧度值。
- ②对机械臂的 6 个电机进行角度调节以及标定。
- ③可以查看机械臂与电脑端是否连接和机械臂整体的使能开关以及整体关节的中值标定。
- ④软件升级，对软件进行升级，并且可以恢复默认设置。
- ⑤进行运动学控制，给定坐标控制机械臂运动。
- ⑥对机械臂的动作进行示教以及动作执行。

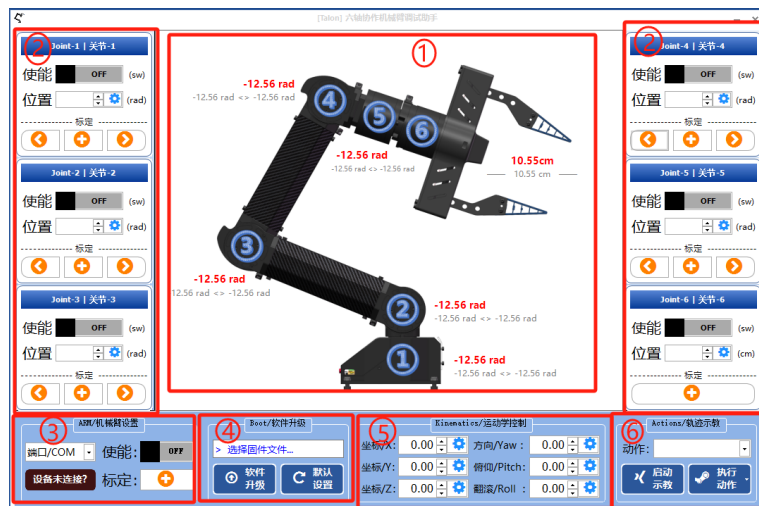


图 6-9 机械臂调试助手软件界面

第三步，连接机械臂。将机械臂的通信线与电脑连接，然后打开上位机软件“[Talon]六轴协作机械臂调试助手”，机械臂会自动与电脑端连接。在软件界面左下角可以查看连接状态。



图 6-10 设备连接成功

第四步，机械臂标定。对机械臂各关节进行最大角度、最小角度以及中值的标定。机械臂关节一到关节五的中值需要在机械臂安装商服机器人之前标定。标定方法：如图，将机械臂完全直立状态下进行标定。



图 6-11 机械臂关节标定

机械臂的最大最小值按照机械臂可活动的范围进行标定。先将机械臂与软件连接然后在软件中间位置可实时查看到机械臂各个关节的弧度值，手动控制机械臂记录每个关节的最大

最小值然后进行标定。

关节六：输入越大数值时爪子开口越大，根据数值调节将爪子刚好闭合时点击

 标定按钮，将爪子闭合状态标定为 0 点。

进行调节时由于不知道爪子初始状态数值，为了避免损坏机械臂可以将当前状态标定为 0 点，然后根据闭合距离进行逐渐减小数值进行调节。

注意：进行调节时建议根据爪子的开合状态进行调节，当调节数值过大或者过小时容易导致机械臂损坏。



图 6-12 机械抓处于闭合状态

如下图红框内要注意铁架与塑料壳的距离不要过近，避免对机械臂造成损坏。

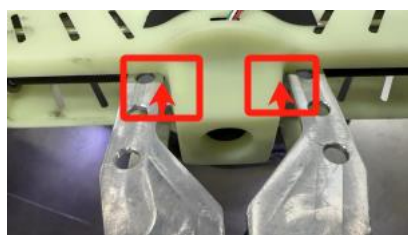


图 6-13 机械臂闭合时注意图示

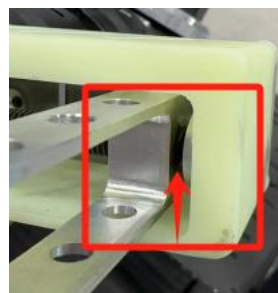


图 6-14 机械臂张开时注意图示

第五步，机械臂动作示教。

(1) 对动作进行演示校准，**伸展动作->收缩动作->放置动作**必须按顺序连续示教。进行示教动作一定要慢，速度过快会导致执行动作时机械臂损坏。为了示教动作的准确及方便，可提前拆除商服机器人后侧结构。具体示教过程可参考工具包提供的“示教视频”。注意：示教连续动作时需要注意动作的连贯性，否则会导致机械臂损坏。（例：在示教收缩动作前必须先示教伸展动作并且连续示教，保证收缩动作的初始位置与伸展动作的结束位置一致）



图 6-15 使能按钮处于关闭状态

(2) 将机械臂放置于初始位置（在执行此示教动作前保证初始位置一致）



图 6-16 伸展动作初始位置



图 6-17 收缩动作初始位置



图 6-18 放置动作初始位置

(3) 在界面右下角中轨迹示教模块，点击下拉框选择需要进行示教的动作，之后点击启动示教操作。



图 6-19 选择示教动作



图 6-20 启动示教

(4) 手动控制机械臂的行进轨迹，最后点击停止示教。



图 6-21 控制机械臂云端

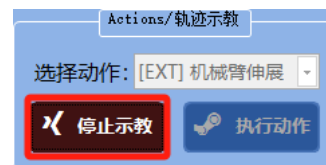


图 6-22 停止示教

第六步，机械臂动作执行。机械臂根据示教动作进行运动，同一个动作不可连续操作。并且，在执行机械臂的动作之前要确保它的初始位姿与示教动作时的初始位姿一致，否则会导致机械臂损坏。(例:在执行收缩动作时要先执行伸展动作，否则收缩动作的初始位姿很难保持与示教时一致)

(1) 将机械臂放置对应示教动作时的初始位置。





图 6-23 伸展动作初始位置



图 6-24 收缩动作初始位置



图 6-25 放置动作初始位置

(2) 在界面右下角中轨迹示教模块，点击下拉框选择需要进行执行的动作，点选完成后，继续点击后面的**执行动作**按钮。



图 6-26 选择动作



图 6-27 执行动作

机械臂会按照示教轨迹自动运行，运行完成后点击**停止动作**。



图 6-28 机械臂自动运行



图 6-29 停止动作

## 步骤 5：智能取餐实操效果运行

因为是单元测试，我们仅测试取餐的过程，也就是商服机器人在餐桌前，抓取食物的行为。在此步骤之前需要对机械臂完成示教工作，否则机械臂不会执行抓取动作，机械臂完成示教工作后，那么有如下对应的实操过程。

第一步，填充缺失代码，完成智能取餐的程序编写。参照之前实训的连接方式，通过连接网线、以太网设置等方式将个人 PC 远程连接商服机器人。将本次实训所提供的缺失代码文件“pick(需填充代码)”，根据提示及实训手册中所讲解的内容完成缺失代码的填充。程序编写完成后，通过（网络映射）共享文件夹的方式根据路径“workspace/sebot-t710-competition/sebot\_catering/src/sebot\_catering/unit/pick.cpp”完成代码源文件的替换。至此我们的程序代码文件已经完成替换，后面可对我们自己编写的程序进行实操验证，注意替换过程中不要将源文件进行直接替换，需先另存，否则很有可能导致程序无法有效运行。



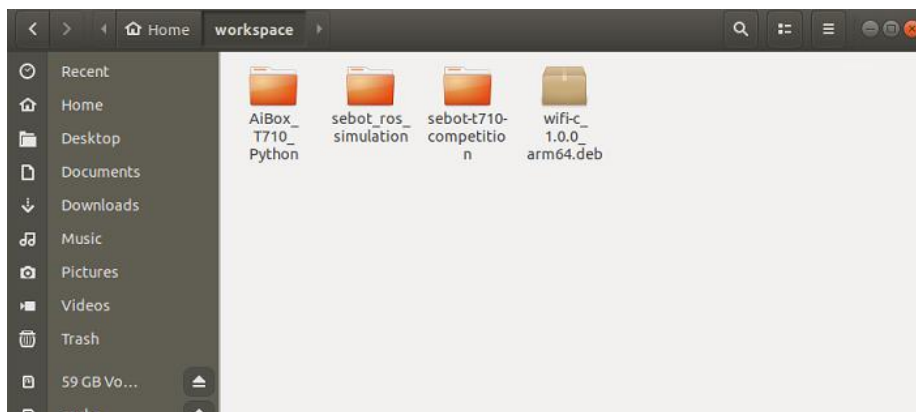


图 6-30 sebot-t710-competition 工程文件所在位置

第二步，确定抓取目标。pick.cpp 是脚本测试代码。可以测试抓取的食物一共有六种，可修改的参数有可乐（LABEL\_AI\_COLA）、咖啡（LABEL\_AI\_COFFEE）、汉堡（LABEL\_AI\_BURGER）、瑞士卷（LABEL\_AI\_CAKE）、奶酪（LABEL\_AI\_CHEESE）、圣代（LABEL\_AI\_SUNDAE）。如果想测试抓取某一种食物，则在下图中的对应位置修改呈对应的参数即可，本次测试我们已经将程序中对应位置的参数调整为咖啡参数，那么本次实操商服机器人则会寻找咖啡食物完成抓取任务。

```

5   while (ros::ok())
6   {
7       if (PICK_SENCE == 1) // 抓取食物场景
8       {
9           if (!run)
10              run = picking.pickupSomething(LABEL_AI_COFFEE);
11      }
12
13      else if (PICK_SENCE == 2) // 放置食物场景
14      {
15          picking.placeFood = 2;
16          if (!run)
17              run = picking.putdownSomething();
18      }
19      else if (PICK_SENCE == 3) // 机械爪跟随食物控制
20      {
21          picking.pickupSomething(LABEL_AI_COFFEE);
22          picking.pickStep = picking.PICK_STEP_AIM;
23      }
24
25      ros::spinOnce(); // 回调话题通信
26      loop_rate.sleep(); // ROS线程周期控制
27  }
28  return 0;
29  }

```

图 6-31 修改 pick.cpp 文件

第三步，将商服机器人摆放至取餐台面前，并且摆放好多种食物。



图 6-32 取餐台上摆好各种食物

第四步，将商服机器人开机启动，可以看到商服机器人与取餐台之间的位置情况。



图 6-33 商服机器人与订单牌子所在位置

第五步，在商服机器人中打开终端。因为程序文件发生了变化，所以需要对整个工程项目进行重新编译，基于商服机器人屏幕及无线键鼠，打开终端，输入指令“cd workspace/sebot-t710-competition/sebot\_catering/”切换到该路径下，因为我们所使用的功能包在 sebot\_catering 中。继续输入编译指令“catkin\_make”。

```
root@paddlepi: ~/workspace/sebot-t710-competition/sebot_catering
File Edit View Search Terminal Help
W: [pulseaudio] main.c: This program is not intended to be run as root (unless -
-system is specified).
N: [pulseaudio] main.c: User-configured server at {6781da706d6a4d8e8d2d8a298d8c5
296}unix:/run/user/0/pulse/native, which appears to be local. Probing deeper.
root@paddlepi:~# cd workspace/sebot-t710-competition/sebot_catering/
root@paddlepi:~/workspace/sebot-t710-competition/sebot_catering# catkin_make
```

图 6-34 切换路径并输入编译指令

```
root@paddlepi: ~/workspace/sebot-t710-competition/sebot_catering
File Edit View Search Terminal Help
k.cpp.o
[ 41%] Building CXX object sebot_catering/CMakeFiles/sebot_ordering.dir/unit/or
der.cpp.o
[ 50%] Building CXX object sebot_catering/CMakeFiles/sebot_catering.dir/src/cat
ering.cpp.o
[ 58%] Linking CXX executable /root/workspace/sebot-t710-competition/sebot_cate
ring/devel/lib/sebot_catering/sebot_arm
[ 58%] Built target sebot_arm
[ 66%] Linking CXX executable /root/workspace/sebot-t710-competition/sebot_cate
ring/devel/lib/sebot_catering/sebot_paying
[ 66%] Built target sebot_paying
[ 75%] Linking CXX executable /root/workspace/sebot-t710-competition/sebot_cate
ring/devel/lib/sebot_catering/sebot_yolo
[ 75%] Built target sebot_yolo
[ 83%] Linking CXX executable /root/workspace/sebot-t710-competition/sebot_cate
ring/devel/lib/sebot_catering/sebot_ordering
[ 91%] Linking CXX executable /root/workspace/sebot-t710-competition/sebot_cate
ring/devel/lib/sebot_catering/sebot_picking
[ 91%] Built target sebot_ordering
[ 91%] Built target sebot_picking
[100%] Linking CXX executable /root/workspace/sebot-t710-competition/sebot_cate
ring/devel/lib/sebot_catering/sebot_catering
[100%] Built target sebot_catering
root@paddlepi:~/workspace/sebot-t710-competition/sebot_catering#
```

图 6-35 编译完成

关闭掉该页面，重新打开一个终端窗口，输入指令“roslaunch sebot\_catering sebo  
t\_picking.launch”启动智能取餐功能。

```
root@paddlepi: ~
File Edit View Search Terminal Help
W: [pulseaudio] main.c: This program is not intended to be run as root (unless -
-system is specified).
root@paddlepi:~# roslaunch sebot_catering sebot_picking.launch
```

图 6-36 智能取餐功能启动

第七步，智能取餐。程序启动后，首先商服机器人会在餐桌前继续调整位姿到适当的位  
置，之后通过商服机器人上的深度摄像头识别桌子上订单对应食物所在的位置。当识别到咖  
啡所在的位置（咖啡为本次实操所调整的食物），机器人调整位姿至对应食物的正向中心位  
置。最后，抬起机械臂，准备抓取。



图 6-37 抬起机械臂

抓取过程中，机械臂中的 RGB 摄像头会反复校准咖啡杯子的位置，保证食物在机械爪的

中心位置，最终将咖啡杯抓起。



图 6-38 咖啡杯子的抓取

第八步，机械臂抓取食物后，会进行收缩动作，至此商服机器人取餐抓取的动作演示完成。

## 4. 拓展思考

如果想调整商服机器人的移动速度，该怎么修改代码文件？应修改哪个代码文件？